

AutoSuivi.tn — Guide d'Implémentation Complet

Version 2.0 — Mai 2026

1. Architecture Finale

Frontend (React Native / Expo)

```
src/
├── core/
│   ├── constants/           # Colors, Spacing, MaintenanceTypes, DeadlineTypes
│   ├── network/
│   │   └── ApiClient.ts     # Axios + JWT + snake_case→camelCase mapping
│   └── domain/
│       ├── entities/        # Vehicle, Maintenance, Deadline, MileageReading, User
│       ├── repositories/    # IVehicleRepository, IMileageRepository, IMaintenanceRe-
│       │                   pository, IDeadlineRepository
│       └── data/
│           ├── datasources/
│           │   └── remote/   # VehicleRemoteDataSource, MileageRemoteDataSource, Main-
│           │               tenanceRemoteDataSource, DeadlineRemoteDataSource
│           │   └── MockDataSource.ts
│           └── repositories/ # VehicleRepositoryImpl, MileageRepositoryImpl, Maintenan-
│               ceRepositoryImpl, DeadlineRepositoryImpl
│       └── services/
│           ├── KmCalculator.ts    # Algorithme de prédiction local
│           └── PredictionService.ts # Service hybride (backend + fallback local)
├── presentation/
│   ├── components/
│   │   # GlassCard, CircularGauge, DeadlineCard, MaintenanceItem, etc.
│   └── context/
│       ├── AuthContext.tsx      # JWT auth state
│       └── VehicleContext.tsx   # State management pour véhicules, maintenance,
│                               deadlines, prédictions
└── app/
    ├── _layout.tsx             # Expo Router (file-based routing)
    ├── index.tsx              # Root layout (AuthProvider + VehicleProvider)
    ├── auth/                  # Redirect screen
    │   └── index.tsx          # Login / Signup
    ├── tabs/                  # Bottom tabs navigation
    │   ├── index.tsx         # Dashboard
    │   ├── vehicles.tsx      # Liste véhicules
    │   ├── mileage.tsx       # Suivi kilométrage
    │   ├── maintenance.tsx   # Maintenance prédictive
    │   └── deadlines.tsx     # Échéances
    ├── vehicles/              # Formulaire véhicules
    │   ├── form.tsx
    │   └── edit.tsx
    ├── maintenance/
    │   └── form.tsx           # Formulaire ajout maintenance
    ├── settings/
    │   └── intervals.tsx     # Intervalles personnalisés
```

Backend (Node.js / Express / MariaDB)

```
autosuivi-api/
├── index.js           # Express app avec Helmet, CORS, Rate Limiting, Morgan
├── .env               # Configuration (DB, JWT, Port)
├── logs/              # Winston logs (error.log, combined.log)
├── src/
│   ├── config/
│   │   └── db.js      # Pool MySQL2 avec promesses
│   ├── middleware/
│   │   ├── auth.js    # JWT verification middleware
│   │   └── validate.js # Joi validation schemas + middleware factory
│   ├── controllers/
│   │   ├── authController.js    # Register, Login (bcrypt + JWT)
│   │   ├── vehicleController.js # CRUD véhicules
│   │   ├── maintenanceController.js # CRUD maintenance + pagination
│   │   ├── deadlineController.js # CRUD échéances (upsert)
│   │   └── predictionController.js # Calcul prédictif côté serveur
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── vehicleRoutes.js
│   │   ├── maintenanceRoutes.js
│   │   ├── deadlineRoutes.js
│   │   └── predictionRoutes.js
│   └── utils/
│       └── logger.js    # Winston logger configuration
```

2. Endpoints Backend Documentés

Authentification

Méthode	Endpoint	Description	Body
POST	/api/auth/register	Inscription	{ name, email, password }
POST	/api/auth/login	Connexion	{ email, password }

Véhicules

Méthode	Endpoint	Description	Body/Params
GET	/api/vehicles/:userId	Liste des véhicules	-
GET	/api/vehicles/detail/:id	Détail d'un véhicule	-
POST	/api/vehicles	Créer un véhicule	{ id, brand, model, year, plate, initial_mileage }
PUT	/api/vehicles/:id	Modifier un véhicule	{ brand?, model?, year?, plate?, current_mileage? }
DELETE	/api/vehicles/:id	Supprimer (cascade)	-

Maintenance

Méthode	Endpoint	Description	Body/Params
GET	/api/maintenance/:vehicleId	Historique maintenance	?page=1&limit=50
POST	/api/maintenance	Ajouter un entretien	{ id, vehicle_id, type, date, mileage, cost?, notes? }
PUT	/api/maintenance/:id	Modifier	{ type?, date?, mileage?, cost?, notes? }
DELETE	/api/maintenance/:id	Supprimer	-

Échéances

Méthode	Endpoint	Description	Body/Params
GET	/api/deadlines/:vehicleId	Liste des échéances	- (inclut days_remaining)
POST	/api/deadlines	Créer/Mettre à jour	{ id, vehicle_id, type, expiry_date }
DELETE	/api/deadlines/:id	Supprimer	-

Prédictions

Méthode	Endpoint	Description	Retour
GET	/api/predictions/:vehicleId	Prédictions complètes	{ prediction, upcomingItems, alerts }

Types valides

MaintenanceType : VIDANGE , FILTRE_HUILE , FILTRE_AIR , FILTRE_HABITACLE , FREINS , PNEUS , COURROIE_DISTRIBUTION

DeadlineType : ASSURANCE , VIGNETTE , VISITE_TECHNIQUE

3. Algorithme de Prédiction

Score d'urgence (0-100)

$$\text{score} = (\text{kmFactor} \times 0.4) + (\text{timeFactor} \times 0.3) + (\text{drivingFactor} \times 0.3)$$

Facteur	Calcul	Poids
km	$\min((\text{kmDepuisDernier} / \text{intervalle}), 1.5) \times 100$	40%
temps	$\min((\text{joursDernier} / \text{max-Jours}), 1.5) \times 100$	30%
conduite	$\text{baseFactor} \times \text{intensity-Factor} \times 50$	30%

Seuils d'urgence

Score	Couleur	Status
0-40	 Vert	OK
41-70	 Orange	Bientôt
71-100	 Rouge	Urgent

Spécificités Tunisiennes

- Huile synthétique : intervalle 10 000 km
- Alerte si > 1 an sans entretien (climat chaud)
- Facteur conduite urbaine : $\times 1.2$ (trafic tunisien dense)

4. Guide de Test

Test Backend (curl)

```
# Health check
curl http://102.204.205.49:3000/health

# Login
curl -X POST http://102.204.205.49/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"test@test.tn","password":"test123"}'

# Validation (doit retourner 400)
curl -X POST http://102.204.205.49/api/maintenance \
  -H 'Content-Type: application/json' \
  -d '{"type":"INVALID"}'

# Prédiction
curl http://102.204.205.49/api/predictions/{vehicleId}
```

Test Frontend

1. **Auth** : Inscription → Connexion → Token stocké
2. **Véhicules** : Ajouter → Lister → Modifier → Supprimer
3. **Kilométrage** : Ajouter lecture → Vérifier stats
4. **Maintenance** : Ajouter entretien → Voir prédictions
5. **Échéances** : Modifier date → Vérifier jauges dashboard
6. **Prédictions** : Vérifier alertes sur le dashboard

5. Sécurité Implémentée

Mesure	Outil	Configuration
Headers HTTP sécurisés	Helmet	Défaut (CSP, HSTS, etc.)
Rate limiting général	express-rate-limit	200 req / 15 min
Rate limiting auth	express-rate-limit	20 req / 15 min
CORS	cors	Toutes origines (mobile)
Validation données	Joi	Tous les endpoints POST/PUT
Auth	JWT	Token 24h, Bearer header
Logs	Winston + Morgan	error.log + combined.log

6. Checklist de Déploiement

Backend

- [x] Helmet installé et configuré
- [x] Rate limiting actif
- [x] Joi validation sur tous les endpoints
- [x] Winston/Morgan pour les logs
- [x] Endpoint de prédiction `/api/predictions/:vehicleId`
- [x] Index SQL optimisés
- [x] PM2 pour process management
- [x] Nginx reverse proxy configuré
- [x] .env avec DB_PASS corrigé

Frontend

- [x] Clean Architecture complète
- [x] Maintenance CRUD → Backend API
- [x] Deadlines CRUD → Backend API
- [x] PredictionService avec fallback local
- [x] Alertes de prédiction sur le dashboard
- [x] eas.json configuré (development, preview, production)

Documentation

- [x] COMPLETE_IMPLEMENTATION_GUIDE.md
- [x] GOOGLE_PLAY_DEPLOYMENT.md
- [x] CHANGELOG.md
- [x] README.md mis à jour